JOUR 4

Workflows et supervision

Jeudi 22 octobre 2026 - 9h10-17h10 - Infrastructure, orchestration et automatisation

Mastère Data Engineering & IA, M2 - EFREI Bordeaux - Alexandre STOPPINI

AU PROGRAMME

- Un bilan quotidien qui décide seul : l'ordre, les relances, l'arrêt, l'issue publiée
- Voir ce qui se passe : métriques, logs, un tableau de bord
- Être prévenu à temps : des alertes, et leurs tests

JOUR 4

Plan de la journée

390 minutes utiles - deux modules - deux ateliers - fin sur un atelier

Créneau	Bloc	Contenu
09h10 - 09h35	Dépôt, QCM	Le dépôt du jour et la correction, puis le QCM sur la journée 3
09h35 - 10h45	Module 7	Un pipeline qui décide seul
10h45 - 11h00	Pause	
11h00 - 12h40	Atelier 7	Le bilan quotidien dans Argo, puis le point de midi
12h40 - 13h40	Déjeuner	
13h40 - 14h25	Module 8	Voir ce qui se passe, être prévenu à temps
14h25 - 16h55	Atelier 8	Alertes, tests, tableau de bord (pause 14h55 - 15h10)
16h55 - 17h10	Bilan	Ce que vous avez construit, la preuve du jour, demain

JOUR 4

Le dépôt du jour, puis le QCM

La correction des ateliers 1 à 6 d'abord, le QCM pendant que tout redémarre

Le QCM, après l'étape 4

- Lien sur la page Moodle du module
- Seul, sans document : 15 minutes
- Après l'envoi : "Afficher le score", lisez les justifications
- Ne compte pas dans la note
- L'étape 5, la vérification : en ouvrant l'atelier 7

Windows, macOS et Linux - depuis $DEPOT

```
# 1. garder votre travail dans l'historique
git add -A
git commit -m "Avant la correction du J4"
# 2. les fichiers du jour
git pull --no-rebase --no-edit cours j4
# 3. la correction (Moodle) : Windows, puis macOS et Linux
Expand-Archive -Force "$HOME\Downloads\correction_J4.zip" .
unzip -o ~/Downloads/correction_J4.zip -d .
git add -A
git commit -m "Correction des ateliers 1 à 6"
git push
# 4. appliquer ; le QCM pendant que ça redémarre
kubectl apply -k deploy/overlays/dev
kubectl apply -k deploy/overlays/prod
# 5. en ouvrant l'atelier 7 : tout doit être OK
./scripts/verifier.ps1 -Atelier 1,2,3,4,5,6
# macOS, Linux : pwsh devant ./scripts/verifier.ps1
```

MODULE 7

Un pipeline qui décide seul

- Pourquoi un orchestrateur de workflows plutôt qu'un cron
- Le DAG, la relance quand elle a un sens, l'issue toujours publiée
- Le calendrier, le suivi, la preuve qu'une étape rejouée ne double rien

MODULE 7

Les mots d'abord

Huit mots pour les workflows, une ligne chacun

Mot	Ce que c'est, en une ligne
Workflow	Une exécution d'un enchaînement de tâches, avec son état : en cours, réussi, en échec
Template	Une étape réutilisable : un conteneur à lancer, ou un enchaînement d'autres templates
DAG	Graphe orienté sans cycle : chaque tâche dit de quelle autre elle dépend
WorkflowTemplate	Un modèle rangé dans le cluster ; on en lance des exécutions
CronWorkflow	Lance un WorkflowTemplate selon un calendrier
retryStrategy	Combien de fois relancer une étape, pour quels échecs, à quel rythme
onExit	Un template exécuté à la fin du workflow, réussite ou échec
Idempotence	Rejouer une étape pour le même jour ne crée aucun doublon

MODULE 7

Le bilan quotidien, ce qu'il produit

Chaque nuit, pour la veille : les trajets par région et par heure, sans donnée personnelle

	Étape	Ce qu'elle fait	Ce qu'elle laisse derrière elle
1	`extract`	Lit les trajets de la veille dans PostgreSQL ; remplace chaque usager par un pseudonyme	`velocite-raw : trips/date=J/trips.csv`
2	`check`	Contrôle le fichier : coordonnée vide, valeur négative, région vide, aucun trajet : refus	`le rapport qualité : trips/date=J/_qualite.json`
3	`aggregate`	Compte par région et par heure : trajets, distance, chiffre d'affaires	`velocite-curated : region_heure.csv` `la table daily_region_stats`
4	`publish`	Déclare le bilan du jour complet, prêt à être lu	`le marqueur _SUCCESS (date, workflow, heure)`
5	`signaler`	Hors de la chaîne : à la fin, à chaque fois, dit si le bilan a réussi ou échoué	`Pushgateway : le dernier statut, l'heure du dernier succès`

MODULE 7

Pourquoi pas un cron et quatre commandes

Une nuit de la v1, puis la même nuit avec un orchestrateur

La v1 : un cron lance les quatre commandes à la suite, sans rien vérifier



2 h 40 : un bilan faux est publié ; personne n'est prévenu

9 h : la finance travaille sur un bilan faux

Avec un orchestrateur : Argo Workflows décide à la place d'un humain



30 s, 60 s, 120 s : la base revient, ou pas

onExit : signaler tourne à la fin quoi qu'il arrive ; ici, elle publie que le bilan a échoué. l'alerte BilanEchoue part vers 2 h 35, pas à 9 h ; aucun bilan faux n'est publié.

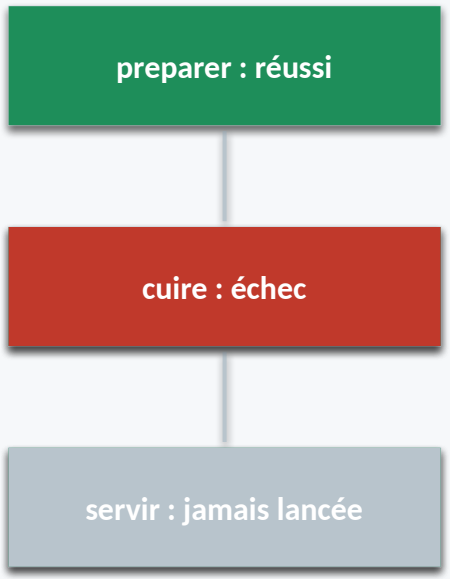
MODULE 7

Un DAG, sur un exemple neutre

Chaque tâche dit de laquelle elle dépend ; une dépendance en échec arrête la suite

YAML - un template de type dag

```yaml
templates:
  - name: recette
    dag:
      tasks:
        - name: preparer
          template: cuisine
          arguments:
            parameters: [{name: action, value: preparer}]
        - name: cuire
          depends: preparer      # attend preparer
          template: cuisine
          arguments:
            parameters: [{name: action, value: cuire}]
        - name: servir
          depends: cuire
          template: cuisine
          arguments:
            parameters: [{name: action, value: servir}]
```



1. Aucune dépendance : démarre tout de suite
2. depends: preparer : attend sa réussite
3. Même template cuisine, un autre paramètre
4. Une autre nuit : cuire échoue, servir ne démarre jamais
5. Le bilan : extract, check, aggregate, publish, la même chaîne

MODULE 7

Relancer, seulement quand ça a un sens

Le code de sortie dit si un nouvel essai peut réussir ; le délai double à chaque essai

YAML - la relance du bilan

```yaml
retryStrategy:
  limit: "3"                # trois relances au plus
  retryPolicy: OnFailure
  expression: "asInt(lastRetry.exitCode) == 75"
  backoff:
    duration: "30s"         # puis 60 s, puis 120 s
    factor: "2"
    maxDuration: "10m"
```

2
1

Code	Ce qu'il dit	Relancer ?
2	donnée invalide	non
75	panne temporaire	oui
78	configuration absente	non

Les codes du batch : en tête de app/services/batch/batch.py

1

essai 1 code 75 → (30 s) → essai 2 code 75 → (60 s) → essai 3 code 0

la base est revenue : la suite part

2

essai 1 code 2

l'expression est fausse : aucune relance, l'échec remonte tout de suite

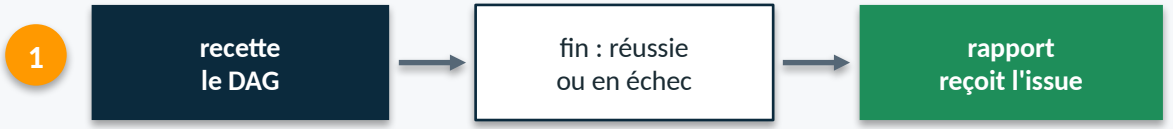
Sans expression : la donnée invalide est relancée trois fois pour rien, l'alerte arrive environ quatre minutes plus tard (30 s + 60 s + 120 s, plus les essais).

Ce qui encadre une exécution

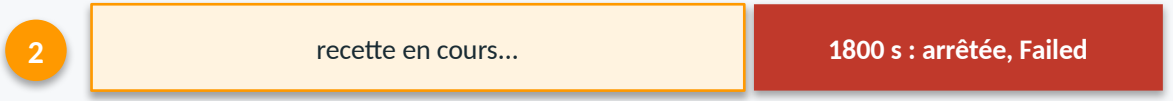
Trois champs au niveau du workflow : ce qui se passe à la fin, combien de temps, combien à la fois

YAML - exemple neutre

```yaml
spec:
  entrypoint: recette
  onExit: rapport                          # 1
  activeDeadlineSeconds: 1800              # 2
  synchronization:                         # 3
    mutexes:
      - name: un-seul-a-la-fois
  templates:
    - name: rapport                        # 1
      container:
        image: mirror.gcr.io/library/busybox:1.37
        command: [echo]
        args: ["{{workflow.status}}"]
```



onExit nomme le template de fin : rapport tourne toujours, et reçoit {{workflow.status}} : Succeeded, Failed ou Error. Pour le bilan, c'est signaler.



Au-delà, le workflow est arrêté en échec ; rapport tourne quand même. Un bilan bloqué finit par se voir.



Le mutex est un verrou nommé, un-seul-a-la-fois : B démarre quand A a fini. Un bilan relancé à la main attend celui de la nuit.

MODULE 7

Le calendrier

Un CronWorkflow lance le modèle selon un calendrier, à l'heure de Paris

YAML - exemple neutre

```yaml
kind: CronWorkflow
metadata: {name: recette}
spec:
  schedules: ["0 7 * * 1"]
  timezone: "Europe/Paris"
  concurrencyPolicy: Forbid
  startingDeadlineSeconds: 600
  successfulJobsHistoryLimit: 2
  failedJobsHistoryLimit: 4
  workflowSpec:
    workflowTemplateRef: {name: recette}
```

1

0	7	*	*	1
minute	heure	jour	mois	jour sem.

chaque lundi à 7 h 00

2 Europe/Paris : suit les changements d'heure

3 Forbid : une exécution tourne encore, on n'en lance pas d'autre

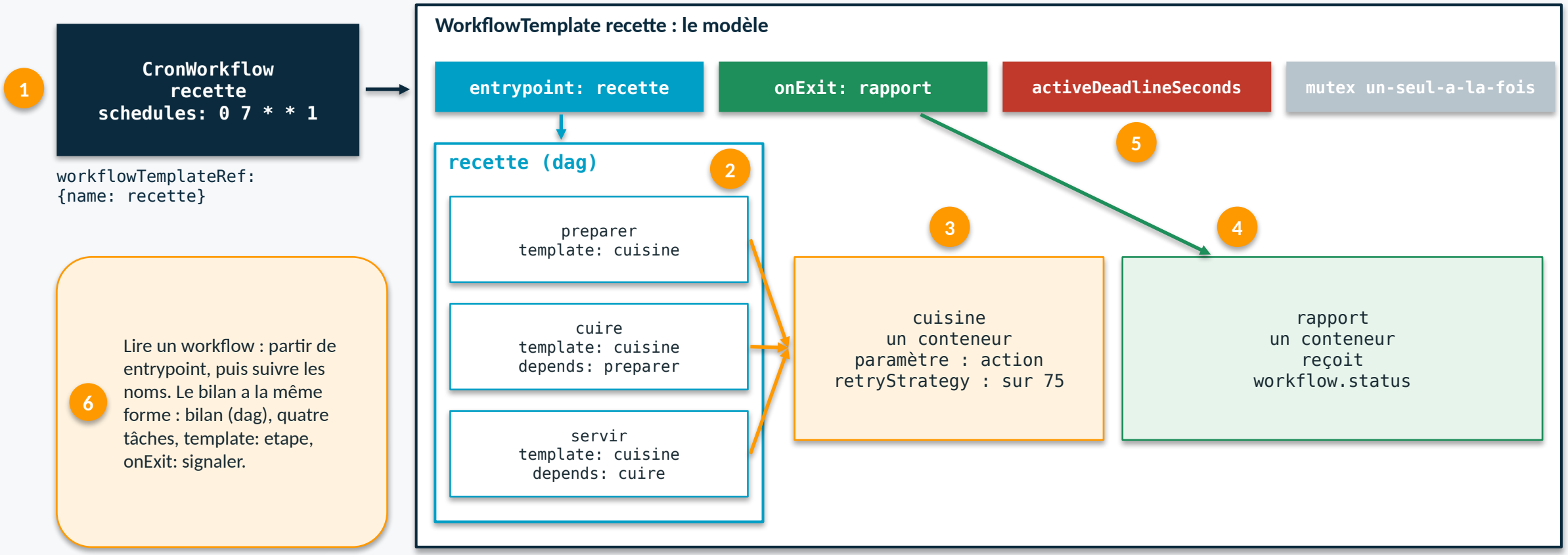
4 Cluster éteint à l'heure dite : rattrapage jusqu'à 10 min, pas au-delà

5 Un historique court, puis le modèle à lancer

6 Deux affichages trompeurs, sans panne : argo lint refuse Europe/Paris sous Windows (le binaire n'a pas la base des fuseaux, le cluster l'a) ; argo cron list affiche NEXT RUN N/A même quand le calendrier tourne. En Argo 4, seul schedules (une liste) est accepté.

Le workflow assemblé, bloc par bloc

Chaque bloc a un nom ; les autres le désignent par ce nom



MODULE 7

Lancer, suivre, enquêter

Le bilan du 12 mars, de bout en bout - et où sont les logs une fois les pods partis

Windows, macOS et Linux - depuis $DEPOT

```
kubectl -n velocite-dev apply -f workflows
argo -n velocite-dev submit --from workflowtemplate/bilan-quotidien -p date=2026-03-12
argo -n velocite-dev logs '@latest' --follow
argo -n velocite-dev list
argo -n velocite-dev get '@latest'
```

1 Pendant l'exécution	2 Après, étape réussie	3 Après, étape en échec
Le pod tourne : argo logs --follow	Pod supprimé (podGC, le ménage d'Argo) : Loki a gardé ses logs, à lire dans Grafana cet après-midi	Le pod reste 7 jours : argo logs, kubectl logs

Profil léger (8 Go) : ni Loki ni Grafana Explore pour les logs ; lisez-les pendant l'exécution.

MODULE 7

Prouver qu'une étape rejouée ne double rien

Le même jour, trois bilans de suite : on compte les lignes du jour après chacun

Windows, macOS et Linux - depuis $DEPOT

```
./scripts/compter-bilan.ps1
# répond : Lignes du 2026-03-12 dans velocite-dev : 33   (macOS, Linux : pwsh devant)
# ce qu'il lance dans le pod postgres :
# SELECT count(*) FROM daily_region_stats WHERE jour = '2026-03-12';
```

1

Remplacer : ce que fait aggregate

DELETE du jour, puis INSERT (transaction)

```
après le 1er bilan : 33
après le 2e bilan  : 33
après le 3e bilan  : 33
```

2

Ajouter : une étape naïve

INSERT seulement, sur une table sans clé

```
après le 1er bilan : 33
après le 2e bilan  : 66
après le 3e bilan  : 99
```

3

Les trois bilans sont Succeeded des deux côtés. Seul le compte montre la différence : un workflow réussi ne prouve rien sur les données.

MODULE 7

Les droits du bilan

Les pods du bilan ont un compte de service : le minimum, rien de plus

Ce qui est fourni

- rbac.yaml : le compte de service bilan, son token dans un Secret
- rbac.yaml : un rôle qui ne permet qu'une chose, écrire le résultat de chaque étape
- bilan-quotidien.yaml : le token aux conteneurs d'Argo, jamais au code du batch
- reseau.yaml (atelier 5) : postgres, garage, Pushgateway, API Kubernetes
- bilan-quotidien.yaml : le durcissement des services, uid 10001, lecture seule

Ce qu'il ne faut pas faire pour gagner du temps

- Donner au bilan le compte par défaut du namespace
- Ouvrir le réseau du bilan pour "que ça passe"
- Monter les secrets en variables d'environnement
- Rendre le compte administrateur du cluster

ATELIER 7

Atelier 7 - Le bilan quotidien dans Argo

95 min jusqu'à 12h35, Si ça coince affiché dès 11h30 - en binôme - Fiche_Atelier7_J4 - mêmes numéros dans Commandes_J4

Consignes, dans l'ordre

1. workflows/bilan-quotidien.yaml : écrire les A FAIRE 1 à 5, puis argo lint
2. Envoyer le modèle, lancer le bilan du 12 mars, le suivre jusqu'au bout
3. BATCH_FAIL_RATE à "0.3" ; refaire 2 ; lire les relances ; remettre "0"
4. La preuve : compter, refaire 2, compter
5. cron-bilan.yaml : chaque nuit à 2 h 30, heure de Paris
6. RUNBOOK.md, Reprise du bilan ; PARAMETRES.md, Le bilan quotidien
7. La vérification : tout OK

Windows, macOS et Linux - depuis $DEPOT

```
# 1. vérifier le fichier, sans cluster
argo lint --offline workflows/bilan-quotidien.yaml
# 2. envoyer, lancer le 12 mars, suivre (Ctrl+C à la fin)
kubectl -n velocite-dev apply -f workflows
argo -n velocite-dev submit --from workflowtemplate/bilan-quotidien -p date=2026-03-12
argo -n velocite-dev watch '@latest'
argo -n velocite-dev get '@latest'
# 3. après avoir changé BATCH_FAIL_RATE : refaire 2
# 4. compter les lignes du 12 mars, avant et après
./scripts/compter-bilan.ps1
# 5. envoyer le calendrier (NEXT RUN N/A : normal)
kubectl -n velocite-dev apply -f workflows
argo -n velocite-dev cron list
# 7. la vérification de l'atelier
./scripts/verifier.ps1 -Atelier 7
# macOS, Linux : pwsh devant chaque ./scripts/...
```

ATELIER 7

Si ça coince

Les blocages relevés pendant la répétition, et leur sortie

Ce qu'on voit	La cause	La sortie
unknown time zone Europe/Paris	argo lint sous Windows	linter le template seul ; le cluster accepte
NEXT RUN N/A	affichage d'argo cron list	lire schedules ; cron get : le dernier passage
argo logs ne montre que signaler	pods réussis supprimés	--follow pendant, Loki après
Code 2, aucune relance	date invalide	voulu : relancer ne sert à rien
Workflow Pending	le mutex attend un bilan en cours	argo list ; laisser finir
argo : requires at least 1 arg(s)	PowerShell retire @latest sans prévenir	les apostrophes : '@latest'
Code 75 en boucle, sans panne injectée	règle réseau batch	sorties vers postgres et garage (atelier 5)
ErrImagePull sur les étapes	image du batch absente	charger-images.ps1 (Dockerfile de lundi)

ATELIER 7

Le point de midi

Cinq minutes : où en êtes-vous ? L'après-midi lit l'issue du bilan

Avant de déjeuner

- Commit et push de ce qui est écrit, même incomplet
- BATCH_FAIL_RATE remis à "0" dans le dépôt
- argo -n velocite-dev list : notez l'état du dernier bilan
- Le cluster peut rester allumé pendant le déjeuner

À 13h40, il faut au minimum

- Le bilan du 12 mars Succeeded, avec son étape signaler
- Les relances vues au moins une fois
- Sinon : le corrigé du fichier de workflow, demandez-le-moi
- Le calendrier et le runbook peuvent se finir pendant l'atelier 8

MODULE 8

Voir ce qui se passe, être prévenu à temps

- Métriques et logs : ce qu'on mesure, où on le lit
- Un indicateur vu par l'usager, et le temps qu'il faut à une alerte
- Une alerte et son test ; un tableau de bord rangé dans le dépôt

MODULE 8

Les mots d'abord

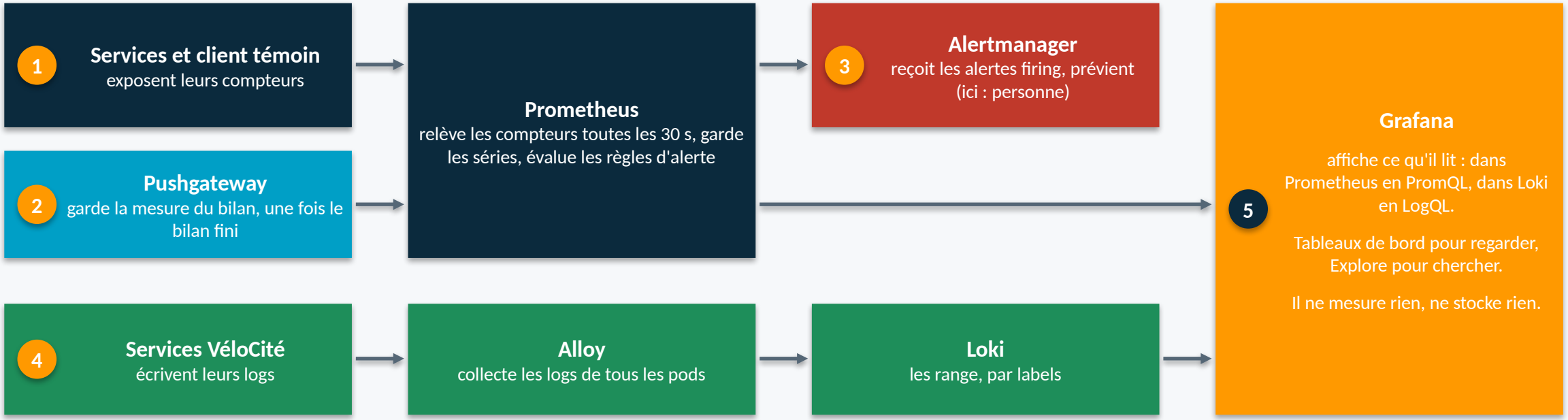
Dix mots pour la supervision, une ligne chacun

Mot	Ce que c'est, en une ligne
Métrique	Une valeur numérique mesurée dans le temps
Compteur	Une métrique qui ne fait que monter ; on regarde sa vitesse
Label	Une étiquette qui découpe une métrique : route, code
PromQL	Le langage de requête de Prometheus
Logs	Les lignes écrites par les services, collectées par Alloy, rangées dans Loki
SLI	Service Level Indicator : la part d'appels qui réussissent, vue de l'usager
SLO	Service Level Objective : l'objectif sur cette part, ex. 99 % sur 30 jours
Budget d'erreur	Ce que l'objectif tolère : 1 % d'échecs pour un SLO à 99 %
Alerte	Une condition vraie pendant une durée (for), qui prévient quelqu'un
Pending, firing	Condition vraie mais pas encore assez longtemps / alerte partie

MODULE 8

Qui fait quoi dans la supervision

Six outils, installés mardi par la couche infra/10-outils : les métriques, puis les logs



6 Profil léger (8 Go) : ni Alloy ni Loki, les logs se lisent avec kubectl logs. Chaque outil s'ouvre sur votre poste par un port-forward : Prometheus sur 9090, Grafana sur 3000.

MODULE 8

PromQL, l'essentiel

Quatre requêtes, de la série brute à la proportion d'erreurs

PromQL - dans Prometheus, onglet Query

```
1 velocite_client_requests_total
2 rate(velocite_client_requests_total[5m])
3 sum by (code) (rate(velocite_client_requests_total[5m]))
4 sum(rate(velocite_client_requests_total{code=~"5..|erreur"}[5m]))
    / sum(rate(velocite_client_requests_total[5m]))
```

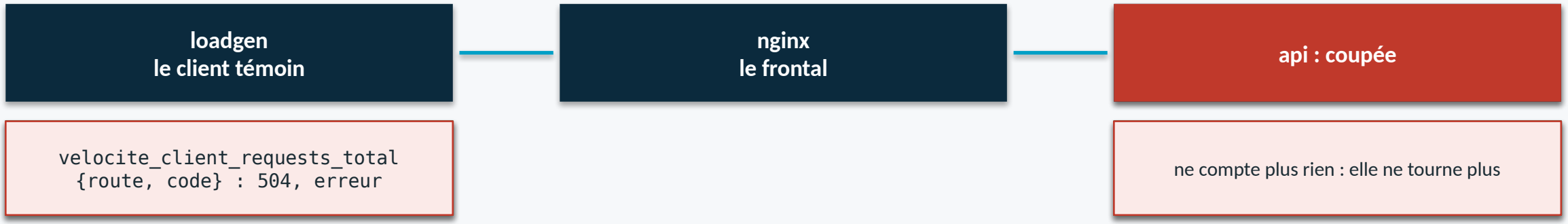
1. Une série par route et par code : des totaux qui ne font que monter, illisibles tels quels
2. Le nombre d'appels par seconde, en moyenne sur les 5 dernières minutes
3. Additionnés par code : combien de 200, 201, 401, 504 par seconde
4. Les erreurs divisées par le total : une proportion, entre 0 et 1

5. Deux règles
- Une proportion se calcule sur des taux (rate), jamais sur les compteurs bruts : ils repartent de zéro à chaque redémarrage
- =~ filtre par expression régulière : 5.. veut dire 500 à 599

MODULE 8

Un indicateur vu par l'usager

Le client témoin compte ses propres appels, y compris ceux qui n'aboutissent pas



1. Tout va bien : les deux côtés comptent les mêmes appels
2. API coupée : seul le client voit la panne, il compte ses erreurs
3. Mesurer par route : tout mélangé, la télémétrie des vélos, plus fréquente, dilue l'API. Coupée, l'API ne fait qu'un tiers d'erreurs au total ; sur ses routes, 100 %

4. Un piège des compteurs : une série naît à son premier événement, déjà à 1. rate() mesure l'écart entre deux relevés : ce 1 n'a pas de relevé avant lui, il n'est jamais compté. Le client témoin crée donc à zéro, dès le démarrage, ses séries d'échec (erreur, 502, 503, 504) pour ses sept routes.

MODULE 8

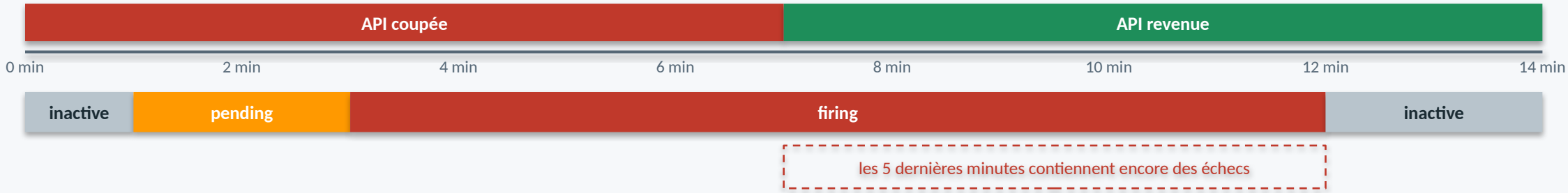
Le temps d'une alerte

Qui fait quoi, puis une coupure de 7 minutes, minute par minute



taux_erreur_5m : la requête 4 de la slide 22, par route, c'est-à-dire la part d'échecs sur les 5 dernières minutes ; for: 2m dans la règle

1. 0 : l'API est coupée ; chaque appel du client témoin vers elle finit en 504
2. +30 s : Prometheus relève les premiers échecs ; sur une route peu appelée, un seul fait déjà 2 %
3. +1 min environ : la règle, évaluée, est vraie : pending ; les 2 min du for commencent
4. +3 min environ : vraie depuis 2 min sans interruption : firing, l'alerte part vers Alertmanager
5. 7 min : l'API revient, mais la fenêtre de 5 min garde ses échecs : la règle reste vraie, toujours firing
6. 12 min : plus aucun échec dans la fenêtre : la règle est fausse, inactive



MODULE 8

Une alerte et son test

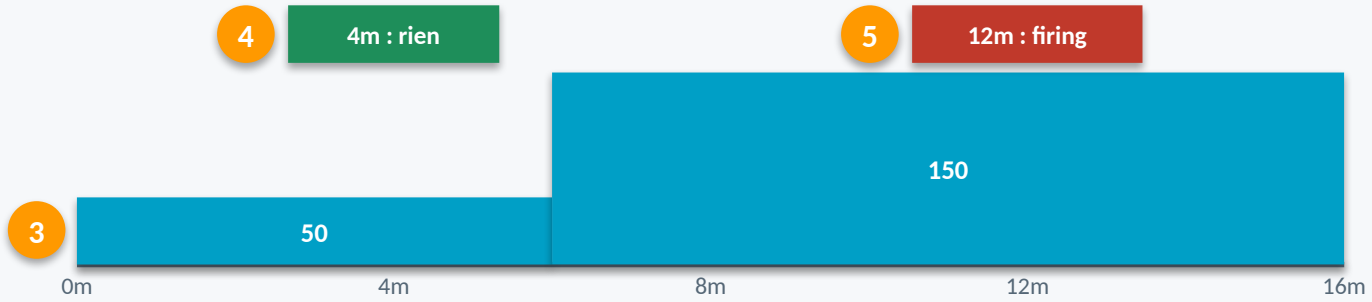
Un test prouve deux choses : elle part quand il faut, elle ne part pas sinon

YAML - une règle (exemple neutre)

```yaml
- alert: FileTropLongue
  expr: file_attente_longueur > 100
  for: 5m
  labels: {severity: avertissement}
  annotations:
    summary: "La file dépasse 100 messages"
    runbook: docs/RUNBOOK.md#file-trop-longue
```

YAML - son test promtool

```yaml
- interval: 1m
  input_series:
    - series: 'file_attente_longueur'
      values: '50x5 150x10'
  alert_rule_test:
    - {eval_time: 4m, alertname: FileTropLongue, exp_alerts: []}
    - eval_time: 12m
      alertname: FileTropLongue
      exp_alerts:
        - exp_labels: {severity: avertissement}
          exp_annotations:
            summary: "La file dépasse 100 messages"
            runbook: docs/RUNBOOK.md#file-trop-longue
```



1. La condition, et combien de temps elle doit tenir
2. Sévérité, résumé, runbook (et une description) : la personne réveillée sait quoi faire
6. Sans le cas à 4m, une alerte qui part tout le temps passerait le test

MODULE 8

Charger, lire, ranger

Les règles et le tableau de bord sont des fichiers du dépôt : ce qu'il faut savoir pour l'atelier 8

1 Charger : du dépôt au cluster

```
observability/regles/alertes.yaml
observability/dashboards/velocite.json
```

Vos alertes et votre tableau de bord, ici.

↓

```
tofu -chdir=infra/10-outils apply
```

OpenTofu les range dans le cluster (des ConfigMaps). Sur un poste léger, avec -var leger=true.

↓

Prometheus et Grafana les relisent : environ 1 min. Attendre avant de conclure qu'une règle manque.

2 Ouvrir Prometheus et Grafana

Un port-forward par outil, chacun dans son terminal : il le bloque.

```
kubectl -n observabilite port-forward
  svc/prometheus-server 9090:80
```

localhost:9090 : onglets Query et Alerts

```
kubectl -n observabilite port-forward
  svc/grafana 3000:80
```

localhost:3000 : utilisateur admin ; le mot de passe sort d'OpenTofu :

```
tofu -chdir=infra/10-outils output
  -raw grafana_mot_de_passe
```

3 Lire les logs : LogQL

Grafana, Explore, source Loki. Entre accolades, les labels ; après |~, un filtre sur le texte.

```
{namespace="velocite-dev",
 level="error"}
```

Les lignes d'erreur de tous les services : ils écrivent error, jamais erreur.

```
{namespace="velocite-dev", app="nginx"}
  |~ "code=5.."
```

Les lignes du frontal en 5xx : les 504 de votre coupure de l'API.

Poste léger, pas de Loki : kubectl logs.

4 Ranger : un panneau construit dans Grafana n'existe que dans Grafana. Inspect, Panel JSON, copier dans la liste panels de velocite.json, puis tofu apply. Sinon, il disparaît à la prochaine reconstruction.

ATELIER 8

Atelier 8, partie 1 - Les alertes et leurs tests

30 min avant la pause, à finir après - en binôme - Fiche_Atelier8_J4 - mêmes numéros dans Commandes_J4

Consignes, dans l'ordre

1. Ouvrir Prometheus ; regarder routes et codes du client témoin, rôles des exports
2. observability/regles/alertes.yaml : une alerte sur objectif, 99 % d'appels réussis, pour chaque route
3. Une alerte qui rend visible la salve d'extraction du compte data science
4. observability/tests/alertes_test.yaml : un test par alerte, elle part quand il faut, pas sinon
5. promtool : check rules, puis test rules, SUCCESS

Windows, macOS et Linux - depuis $DEPOT

```
# 1. Prometheus sur http://localhost:9090
#    (ce terminal reste bloqué : un second pour la suite)
kubectl -n observabilite port-forward svc/prometheus-server 9090:80
# 2 à 4 : écrire dans VS Code
# 5. vérifier, puis tester les règles
./scripts/promtool.ps1 check rules observability/regles/alertes.yaml
./scripts/promtool.ps1 test rules observability/tests/alertes_test.yaml
# macOS, Linux : pwsh devant ./scripts/promtool.ps1
```

ATELIER 8

Grafana, concrètement

Ce que vous verrez cet après-midi : un tableau, ses panneaux, et le vôtre à ajouter



1. Un tableau de bord : une grille de panneaux. Un panneau, c'est une requête, plus une façon de l'afficher : courbe, chiffre, table.

2. Le menu Environnement est une variable, $namespace, remplacée dans chaque requête : le même tableau pour velocite-dev et velocite-prod.

3. Votre panneau se fabrique dans un tableau de brouillon : la requête en mode Code, Legend {{route}}, Unit Percent (0.0-1.0).

4. Explore : une requête sans tableau, pour chercher. Source Prometheus en PromQL, ou Loki en LogQL pour les logs.

5. Ranger : Inspect, Panel JSON, collé dans velocite.json, tofu apply. Ce tableau vient d'un fichier : Grafana refuse de l'enregistrer.

ATELIER 8

Atelier 8, partie 2 - Provoquer, tracer, documenter

100 min après la slide Grafana, Si ça coince compris - en binôme - Fiche_Atelier8_J4 - mêmes numéros dans Commandes_J4

Consignes, dans l'ordre

1. Charger vos règles ; une minute plus tard, onglet Alerts de Prometheus
2. Couper l'API de dev 7 minutes ; noter l'heure de pending, puis de firing
3. Pendant l'attente : la section du runbook de votre alerte
4. Remettre l'API ; noter le retour à inactive ; diff vide
5. Grafana : un panneau de disponibilité par route, copié dans velocite.json, rechargé
6. RUNBOOK une section par alerte, et Service indisponible ; ARCHITECTURE La supervision ; PARAMETRES Les alertes
7. La vérification : tout OK

Windows, macOS et Linux - depuis $DEPOT

```
# 1. charger les règles (poste léger : -var leger=true)
tofu -chdir=infra/10-outils apply
# 2. couper l'API, noter l'heure
kubectl -n velocite-dev scale deployment/api --replicas=0
# 4. au bout de 7 minutes : remettre ; diff vide
# Windows : le diff de Git d'abord (Commandes_J4, étape 4)
kubectl -n velocite-dev scale deployment/api --replicas=1
kubectl diff -k deploy/overlays/dev
# 5. Grafana sur http://localhost:3000, compte admin
kubectl -n observabilite port-forward svc/grafana 3000:80
tofu -chdir=infra/10-outils output -raw grafana_mot_de_passe
# 5. puis recharger le tableau : refaire 1
# 7. la vérification de l'atelier
./scripts/verifier.ps1 -Atelier 8
# macOS, Linux : pwsh devant ./scripts/verifier.ps1
```

ATELIER 8

Si ça coince

Les blocages relevés pendant la répétition, et leur sortie

Ce qu'on voit	La cause	La sortie
Règles absentes après apply	relecture toutes les minutes	attendre une minute
Alerte jamais firing	règle non chargée, ou expression vide	onglet Alerts ; l'expression dans Query
Firing après le retour	les échecs restent 5 min dans la fenêtre	attendre : ce n'est pas une fausse alerte
Grafana : Cannot save provisioned dashboard	tableau venu d'un fichier	Inspect, Panel JSON, dans la liste panels
LogQL : 0 ligne	error, pas erreur ; nginx écrit level=info	level="error" ; pour nginx |~ "code=5.."
Test promtool en échec sur les annotations	texte différent d'un caractère	copier-coller depuis la règle
tofu apply installe Loki sur un poste léger	variable leger oubliée	-var leger=true

BILAN

Ce que vous avez construit depuis lundi

Quatre jours, cinq étages, et pour chacun une preuve qui se rejoue

LUNDI	MARDI	MERCREDI	JEUDI MATIN	JEUDI APRÈS-MIDI
Des images sûres - L'inventaire de la plateforme - Multi-stage, digest, non root - La CI : build, Trivy	**L'infra en code** - OpenTofu : plan, apply, state - Le cluster et ses outils - dev : fermé, secrets générés	**VéloCité sur Kubernetes** - Une règle réseau par service - Base, overlays dev et prod - Une version qui casse, undo	**Le bilan qui décide** - DAG, relances, issue publiée - Un calendrier, chaque nuit - Rejouer ne double rien	**Être prévenu à temps** - Alertes testées - Une vraie panne, ses heures - Un panneau rangé dans le dépôt
Preuve : la CI verte sur main	Preuve : tofu plan : No changes	Preuve : smoke vert ; diff vide	Preuve : 33 lignes, trois fois	Preuve : promtool : SUCCESS

BILAN

La preuve du jour

Ce que vous savez prouver ce soir, et à qui

Windows, macOS et Linux - depuis $DEPOT

```
argo -n velocite-dev list
# le bilan du 12 mars : Succeeded
argo -n velocite-dev cron list
./scripts/promtool.ps1 test rules observability/tests/alertes_test.yaml
# SUCCESS
kubectl diff -k deploy/overlays/dev
# et les trois heures de l'alerte, notées
git add -A
git commit -m "Ateliers 7 et 8"
git push
# macOS, Linux : pwsh devant ./scripts/promtool.ps1
```

En une phrase, pour la directrice financière

"Le bilan de la nuit se refait seul quand la base a un hoquet, sans jamais compter deux fois. Si le site tombe, l'alerte part en quelques minutes, avant que les clients n'appellent."

Les trois heures notées cet après-midi prouvent que l'alerte part. Personne n'est encore notifié : écrivez-le comme une limite, et gardez vos heures pour le 30 novembre.

BILAN

Demain : redéployer, puis aller plus loin

Vendredi 23 octobre - une commande, la preuve qu'il ne reste rien à faire

Ce qui arrive

- Un objectif de service, et ce qu'on fait de son budget d'erreur
- up.ps1 complet : la plateforme renaît en une commande
- La preuve qu'il ne reste rien à faire : plan et diff vides
- L'après-midi : la suite, précisée demain matin

Ce soir, cinq minutes

- Commit et push avant de partir : votre version est celle évaluée ; sans commit, rien n'est évalué
- BATCH_FAIL_RATE vaut "0"
- Les trois heures de l'alerte notées dans le dépôt
- Pas de down.ps1 : demain repart de votre plateforme